CMT 201 LOGIC CIRCUITS

Inroduction

Analog system

The physical quantities or signals may vary continuously over a specified range.

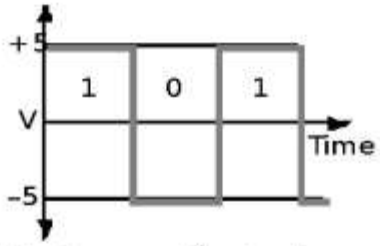
Digital system

The physical quantities or signals can assume only discrete values. Hence greater accuracy.

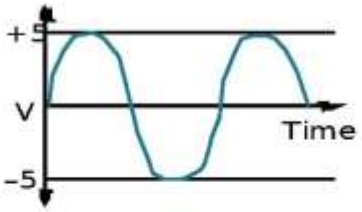
An **information variable** is represented by **physical quantity**.

A **binary signal**, the most common digital signal, is a signal that can take only one of two discrete values and is therefore characterized by transitions between two states.

For digital systems, the variable takes on **discrete values**. **Two level,** or **binary** values are the most prevalent values. Binary values are represented abstractly by: **Digits 0 and 1.** Words (symbols) **False (F) and True (T)**, Words (symbols) **Low (L) and High (H)**, and words **On and Off.**



Digital: only assumes discrete values



Analog: values vary over a broad range continuously

Hence binary values are represented by values or ranges of values of physical quantities.

Logic Circuits

A logic circuit is a special case of a digital system that recognises, processes and outputs only two discrete (voltage) values, high (1) and low (0)

Digital computer

A digital computer stores data in terms of **digits (numbers)** and proceeds in **discrete** steps from one state to the next.The states of a digital computer typically involve **binary digits** which may take the form of the presence or absence of magnetic markers in a storage medium , on-off switches or relays. In digital computers, even letters, words and whole texts are represented digitally.

Digital Logic is the basis of electronic systems, such as computers and cell phones. Digital Logic is rooted in **binary code**, a series of zeroes and ones each having an opposite value. This system facilitates the design of electronic circuits that convey information, including **logic gates**. Digital Logic gate functions include **AND, OR** and **NOT**. The value system translates **input signals** into **specific output.** Digital Logic facilitates computing, robotics and other electronic applications.

Logic gates are physical devices that can be used to implement **logic functions**. They control the flow of signals from the inputs to the single output.

Digital Logic Design is foundational to the fields of electrical engineering and computer engineering. Digital Logic designers build complex electronic components that use both electrical and computational characteristics. These characteristics may involve power, current, logical function, protocol and user input. Digital Logic Design is used to develop **hardware**, such as **circuit boards** and **microchip processors.** This hardware processes user input, system protocol and other data in computers, navigational systems, cell phones or other high-tech systems.

Digital Computers: Imply that the computer deals with digital information, i.e., it deals with the information that is represented by binary digits

Exercise

Q1: State two reasons why almost all digital circuits are really logic circuits.

Advantages of Digital Systems

Digital systems are extensively used for most practical applications in our modern world. They have the advantages of greater flexibility in computation, ease of designing using simple, reusable building blocks, greater precision and miniaturization.

Representations of a Digital Design

Truth tables

A truth table is a tabular summary of all possible outputs of a logic gate, given all possible input values. Truth tables are very useful in defining logic functions.

Gates

Gates are simplest digital logic circuits, and they implement basic

logic operations (functions).

They are designed using transistors and they are used to build more complex circuits that implement more

complex logic functions.

Logic gates are physical devices that can be used to implement logic functions. They control the flow of signals from the inputs to the single output. Logic gates can have an arbitrary number of inputs. The rules that define a logic function are often represented in tabular form by means of a truth table.

OR Gate

If either X or Y is true (1), then Z is true (1)

OR gate

X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

Truth table

AND Gate

If both X and Y are true (1), then Z is true (1)

AND gate

X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

Truth table

Exercise
Q2: Draw a symbol and truth table for a NOT Gate (inverter)

An example of a Logic Design

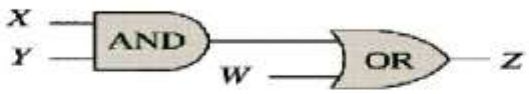
Determine the combination of logic gates that exactly implement the required logic function.

Statement: The output Z shall be logic 1 only when condition (X =1 AND Y =1) OR (W = 1) occurs and shall be logic 0 otherwise.

Solution

X	Y	W	Z
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Solution using logic gates

Boolean Algebra

The **mathematics associated with the binary number system** (and with the more general field of logic) is called **boolean** (George Boole, English Mathematician, circa 1850).

The variables in a boolean, or logic, expression can

take only one of two values, 0 (false) and 1 (true).
Analysis of logic functions (functions of boolean variables) can be carried out in terms of truth tables.

Boolean Algebra and Logic Circuits

The basis of **boolean algebra** lies in the operations of **logical addition,** or the **OR operation,** and **logical multiplication,** or the **AND operation.**

NB. To evaluate logic expressions, make frequent use of truth tables. A **set of rules** will facilitate this task. The following **set of rules and identities** can be used to simplify logic expressions.

Basic Laws of Boolean Algebra

- **Identity laws: A + 0 = A**

  A*1=A

- **Inverse laws: A + A = 1**

  A*A=0

- **Zero and one laws: A + 1 = 1**

A*0=0

- **Commutative laws: A + B = B+A**

  A*B=B*A

- **Associative laws: A + (B + C) = (A + B) + C**

$$A * (B * C) = (A * B) * C$$

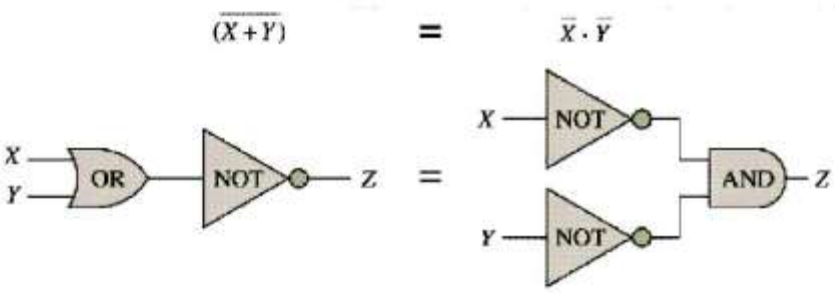
- **Distributive laws : $A * (B + C) = (A * B) + (A * C)$**

$$A + (B * C) = (A + B) * (A + C)$$

DeMorgan's Theorems: State a very important property of logic functions: **Any logic function can be implemented by using only OR and NOT gates, or only AND and NOT gates.**

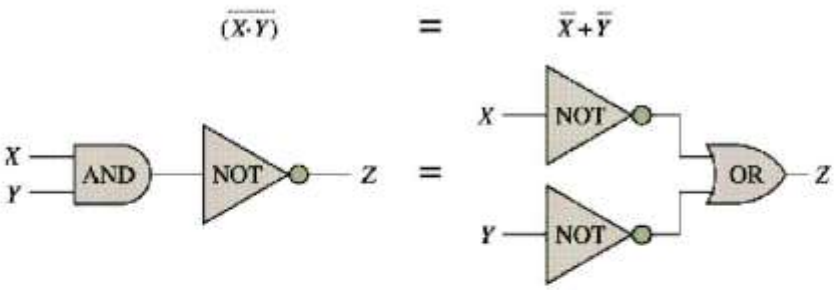
- **DeMorgan's laws: $(A + B) = A * B$**

$$(A * B) = A + B$$



Truth table

X	Y	$Z = \overline{(X+Y)} = \overline{X} \cdot \overline{Y}$
0	0	1
0	1	0
1	0	0
1	1	0

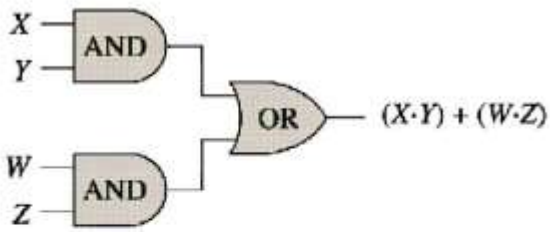


Truth table

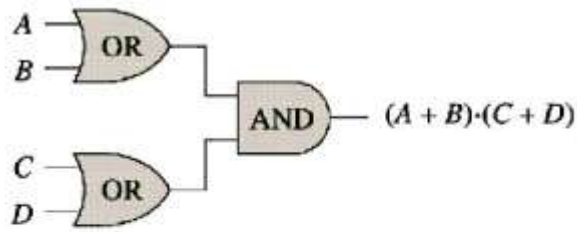
X	Y	$Z = \overline{(X+Y)} = \overline{X} + \overline{Y}$
0	0	1
0	1	1
1	0	1
1	1	0

The **importance** of DeMorgan's Laws lies in the statement of the **duality** that exists between AND and OR operations i.e. any function can be realized by just one of the two basic operations, plus the compliment operation. This gives rise to **two families of logic functions**:

- Sums of Products
- Products of Sums



Sum-of-products expression
$(X \cdot Y) + (W \cdot Z)$

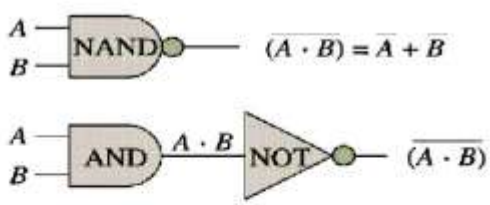


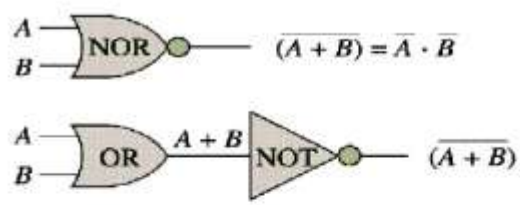
Product-of-sums expression
$(A + B) \cdot (C + D)$

NB. Any logical expression can be reduced to one of these two forms. Although the two forms are equivalent, it may well be true that one of the two forms has a simpler implementation i.e. **fewer gates.**

NAND and NOR Gates

In addition to the AND and OR gates, the complimentary forms of the gates, called **NAND** and **NOR**, are commonly used in practice. It is **important to note that,** by DeMorgan's Laws, **the NAND gate performs a logical addition on the compliments of the inputs**, while the **NOR gate performs a logical multiplication on the compliments of the inputs**. Functionally, then, any logic function could be implemented with either NOR or NAND gates only.





A	B	$\overline{A}$	$\overline{B}$	$\overline{(A \cdot B)}$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

NAND gate

A	B	$\overline{A}$	$\overline{B}$	$\overline{(A + B)}$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

NOR gate

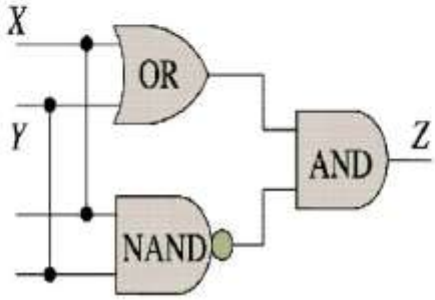
Equivalence of NAND and NOR gates with AND and OR gates

XOR (exclusive OR) Gate

X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	0

Truth table

Realization of an XOR Gate



Designing of Logic Networks

A sequence of steps to follow to apply combinational logic to a real electronic problem:

1. Define the problem in words.
2. Write quasi-logic statements in English that can be translated into Boolean expressions.
3. Write the Boolean expressions.
4. Simplify and optimize the Boolean expressions, if possible.
5. Write an all-AND, all-NAND, all-OR, or all-NOR realization of the circuit to minimize the number of required logic IC gates.
6. Draw the logic schematic for the electronic realization.

Karnaugh Maps and Logic Design

More than one solution is usually available for the implementation of a given logic expression. Some combinations of gates can implement a given function **more efficiently** than others. To be assured of

having chosen the most efficient realization, **a Karnaugh Map** can be used to describes all possible combinations of the variables present in the logic function of interest.

A Karnaugh Map **consists of 2N cells**, where N is the number of logic variables. Row and column assignments are arranged so that all **adjacent terms change by only one bit**.

For example:

The Karnaugh Map provides an immediate view of the values of the function in graphical form.

	$\overline{X}\overline{Y}$	$\overline{X}Y$	XY	$X\overline{Y}$
$\overline{Z}$	0	1	1	0
Z	0	1	1	0

Karnaugh map

X	Y	Z	Desired Function
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Truth table

Example

Use a four-variable logic function to explain how Karnaugh Maps can be used directly to implement a logic function.

X	Y	Y	Z	Desired Function
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Truth table for four-variable expression

	$\overline{W}\cdot\overline{X}$	$\overline{W}\cdot X$	$W\cdot X$	$W\cdot\overline{X}$
$\overline{Y}\cdot\overline{Z}$	1	0	0	0
$\overline{Y}\cdot Z$	1	1	0	1
$Y\cdot Z$	0	0	1	0
$Y\cdot\overline{Z}$	0	1	0	1